AutoScript xT Technical Note

4022 262 70291

FEI COMPANY™
Tools for Nanotech™

Trademark Acknowledgments

FEI and the FEI logo are registered trademarks of FEI Company.
DualBeam, Sidewinder, Tools for Nanotech, and Vectra are trademarks of FEI Company.
FrameMaker is a registered trademark Adobe Systems Incorporated.
Windows is a registered trademark of Microsoft Corporation
Viton is a registered trademarksof DuPont Dow Elastomers L.L.C.
Other product and company names mentioned herein may be the trademarks of their respective owners.

Principal Contributors

Jaco Moonen

Technical Publications

Mike Hayles

Copyright © 2005 by FEI Company

The information and materials contained herein are confidential and proprietary to FEI Company.
They are provided for your organization's internal use on a need-to-know basis.
They cannot be duplicated or disseminated for any third party without the express consent of FEI Company.

The following notice applies to the U.S. Government and other purchases with federal funds:

Limited Rights

Contractor Name: FEI Company
Contractor Address: 5350 NE Dawson Creek Drive, Hillsboro, OR 97124

The Government's rights to use, modify, reproduce, release, perform, display, or disclose these technical data are restricted to those rights specified in DFARS 252.227-7015(b)(2), FAR 52.227-14(g)(2)(Alternate II) and FAR 12.211.
Any reproduction of technical data or portions thereof marked with this legend must also reproduce the markings.
Any person, other than the Government, who has been provided access to such data, must promptly notify the above named Contractor.

AutoScript xT Technical Note

Overview **1**
What is Scripting? 1
Script Structure 1
Running Scripts 2
Information about this Document 3

Commands **4**
Complete List of Commands 4
Patterning and Milling Commands 4
Imaging Commands 7
Cognex VisionPro Tool Commands 8
Control Beam Commands 8
Ion Column Commands 9
Electron Column Commands 10
Stage Commands 11
Detector Commands 11
Miscellaneous Commands 12
Math Commands 14

Variables **15**
Using Expressions 15
User-Defined Variables 15
Result Variables 16
Intrinsic Variables 17
Automation Program Variables 17

Writing a Script **18**
Example Scripts 18
Finding a Feature 18
Forms of the Match Command 18
Examples of the Match Command 19
Automating Milling 21
Defined Dimensions 21
Stream or Pattern Files 22
Capturing Images 22
Recording Data 22
Producing a Dialog Box 23

Troubleshooting **24**
Causes of Imaging Errors 24
Causes of Program File Errors 24
Causes of Recognition Errors 24
Causes of “Invalid ASCII” Error Messages 24

Tips **25**

CONFIDENTIAL—FEI Limited Rights Data

FEI COMPANY™

Clearing Patterns 25
Improving Milling Performance 25
Enhancing Recognition Capabilities 25

An Alphabetical list of xT Commands 26

Index 29

CONFIDENTIAL—FEI Limited Rights Data

AutoScript xT

Overview

AutoScript™ is a powerful yet easy-to-learn language that can automate many functions of FEI® workstations. You can write scripts that control milling, scanning, image recognition, stage movement, electron beam and ion beam settings, gas injection, and other functions.

What is Scripting?

A script is a set of directions that can be executed without user interaction. Think of a script as a recipe. It contains a list of instructions, called commands, and a list of ingredients, called parameters. Commands tell the computer what to do with the parameters. A script may contain just a few commands or an entire series of commands.

A parameter is an item of information; it can be a value or a variable. Variables are names that stand for a value. For example, you can use a variable to represent numeric data, text, or graphics, rather than enter data directly into a program. Then, when the program is executed, the variables are replaced with real data. This allows the same program to process different sets of data.

Two other terms used in scripting are statement, such as an if, then logic statement, and expression, which is a group of symbols representing a value.

Script Structure

AutoScript scripts are stored as text files. These text files can be created and modified with any simple text editor such as Microsoft® WordPad. A “parent” script can call any number of additional “child” scripts, which, after execution, return control to the initial “parent” script.

NOTE	The available commands in AutoScript depend on the version of xT.

Make your scripts modular by defining a parent script and separate child scripts for each major function. With separate scripts it is easy to create new parent scripts and modify child scripts as needed. You can name a script with any valid Windows® file name andextension, but a more useful convention is shown below

:

TABLE 1 Script Naming Conventions

File Extension	Script Type
.ini	Initialization data scripts
.mil	Milling scripts
.psc	Processing and control scripts. Use this for the parent scripts.
.rcg	Image recognition scripts
.sps	Subscripts. Use this for miscellaneous child scripts.

Running Scripts

AutoScript scripts must be used in conjunction with one of the FEI automation control applications as shown in *Table 2*. The application used will depend on the system and your needs. In general, scripts written for one application can be run with any other. For more information about running scripts, refer to the documentation for the specific application you are using.:

TABLE 2 FEI Automation Control Applications

Program	Where Used
AutoFIB™	General purpose automation program in which locations are not in a regular array. Applications include TEM sample
RunScript	Runs scripts; useful for setting up more complex operations and for testing parts of an automation process.

Information about this Document

This document describes the syntax of the AutoScript commands and parameters and shows examples of command sequences that perform specific functions. The syntax controls the words the script understands, the meaningful combinations of words, and the necessary punctuation.

The following table describes the conventions used in documenting the syntax of the scripting language.

TABLE 3 Conventions for Documenting Software Syntax

Convention	Meaning	Example
bold	Elements that appear as bold in the syntax must be typed exactly as they appear.	**ABC U** Type the command and parameter exactly as shown.
italic	Italic font placeholders represent information the user must supply. This information may be a variable, a string, or a number.	*uservariable* Type the appropriate information, usually for a parameter.
{ }	Braces indicate a set of choices from which the user must choose one. The user does not type the braces or commas.	**ABC** {**2**,**4**,**8**} Type the characters ABC followed by a space, followed by either the character 2, 4, or 8.
[ ]	Brackets indicate optional items. The user does not type the brackets. Brackets can be nested.	• **ABC** [*integer*] Type the characters ABC followed by a space, followed optionally by an integer. • **ABC** [*string*[**,** *integer*]] Type the characters ABC followed by a space, followed optionally by a string, that can be followed by a comma and an integer.
|	A pipe separates two mutually exclusive choices. The user types one of these choices, not the symbol.	**ABC** *integer*|*filename* Type the characters ABC followed by a space, followed by either an integer or the name of a file.
…	An ellipsis indicates that a parameter can be repeated several times in a command line. The user types only the information, not the ellipsis.	**ABC** *filename*[**,** *integer*…] Type the characters ABC, followed by a space, followed by a filename, followed optionally by as many comma-integer sequences as desired.

Commands

This section lists and describes all the commands supported by AutoScript.

Complete List of Commands

A complete list of commands and relative to xT can be found on *page 26* of this document.

Following are tables containing a separate grouping of those commands with fuller description.

Patterning and Milling Commands

The units of the patterncommand parameters are in microns, with the origin at the center of the screen (+X right, +Y up).

See also *"Automating Milling" on page 21*.

TABLE 4 Patterning and Milling Commands

Command	Description
bitmap *x1, y1, x2, y2, filename*	Creates a bitmap pattern from a file in the script directory.
box *x1*, *y1*, *x2*, *y2*	Creates a box pattern. (*x1*, *y1*) and (*x2*, *y2*) define the upper left (or NW) and lower right (or SE) corners, respectively. The system assigns an index value for this pattern and sets the value in the result variable **patkey**.
circle *xcenter*, *ycenter*, *r1*, *r2*	Creates a circle pattern designated by the center (*xcenter*, *ycenter*), inner radius (*r1*), and outer radius (*r2*). The system assigns an index value for this pattern and sets the value in the result variable **patkey**.
clear	Clears all milling patterns currently defined.
deletepattern ***key***	Deletes the specified pattern.
getgisfromname *gis-name, gisno*	Gets the GIS number from the name, e.g. "getgisfromname Pt dep, ptgis"
getismilling *millstate*	Returns a value that indicates the milling state: 1 indicates milling and 0 indicates not milling.
getparallelmode *mode*	Returns the milling pattern order. The *mode* values are: 0 Serial milling 1 Parallel milling
getpatbeam *beamtype*	Returns a value that indicates the beam selected for milling the patterns: 0 indicates the ion beam and 1 the electron beam
getpatternrect ***x1*, *y1*, *x2*, *y2*, *key***	Returns the pattern bounding rectangle specified by the index *key*. (*x*1, *y*1) and (*x*2, *y*2) are the upper left (or NW) and lower right (or SE) corners, respectively. The dimensions are in microns, where the center of the screen is (0,0).
getrotation ***angle*, *key***	Returns the pattern rotation for the pattern specified by the index *key*. *angle* is in degrees.

TABLE 4 Patterning and Milling Commands

Command	Description
insert *GISnumber*	Inserts the specified GIS needle. Use with the **retract** command. The input range is 1 to 5.
insertbyname *gisname*	Insert a GIS based on name, e.g. “insertbyname Pt dep”.
line *x1*, *y1*, *x2*, *y2*	Creates a line pattern. (*x*1, *y*1) and (*x*2, *y*2) define the endpoints of the line. The system assigns an index value for this pattern and sets the value in the result variable **patkey**.
mill	Mills all patterns currently defined. Script execution waits until milling is complete.
millnowait	Mills all patterns currently defined, but script execution continues, allowing other commands to be run in parallel with milling.
obox *x1*, *y1*, *x2*, *y2*	Creates an open box (outline) pattern. *(x*1, *y*1) and (*x*2, *y*2) define the upper left (or NW) and lower right (or SE) corners, respectively. The system assigns an index value for this pattern and sets the value in the result variable **patkey**.
patternfile *patfilename*	Loads into memory the specified pattern file (*.ptf*) to be milled. (The file is located in the same directory as the parent script, or the path can be specified with the file name.) Uses the default xT application file or the last used application file.
polygon *x1*, *y1*, *x2*, *y2*, *x3*, *y3*, *x4*, *y4*	Creates a polygon pattern. (*x1*, *y1*) ... (*x4*, *y4*) define the four corners of the polygon area. The system assigns an index value for this pattern and sets the value in the result variable **patkey**.
retract *GISnumber*	Retracts the specified GIS needle.
retractbyname *gisname*	Retract a GIS based on name, e.g. “retractbyname Pt dep”.
setapplicationfile *applicationfile, key*	Sets the application file of the pattern specified by the index *key.*
setparallelmode *mode*	Sets the order in which multiple milling patterns will be milled. The *mode* values are: 0 Serial milling: patterns will be milled one after the other. 1 Parallel milling: all patterns will be milled at the same time.
setpatbeam *beamtype*	Sets the patterning beam for all patterns created: 0 indicates the ion beam and 1 the electron beam.
setpatinfo *z, applicationfilename*	Selects the milling depth and application file used by the milling pattern. (The file is located in the same directory as the parent script, or the path can be specified with the file name.) Use before patterns are defined. See *Figure 2, “Milling Script Example,” on page 21*.
setpatternrect ***x1*, *y1*, *x2*, *y2*, *key***	Sets the pattern bounding rectangle for the pattern specified by the index *key. (x*1, *y*1) and (*x*2, *y*2) define the upper left (or NW) and lower right (or SE) corners, respectively. The dimensions are in microns, where the center of the screen is (0,0). (Use this command to manipulate the outer rectangle for different types of patterns. It is usually used with open box, filled box, cleaning cross section, and regular cross section.) The system assigns an index value for this pattern and sets the value in the result variable **patkey**.

TABLE 4 Patterning and Milling Commands

Command	Description
setrotation *angle, key*	Sets the pattern rotation for the pattern specified by the index *key. angle* is in degrees.
streamfile *filename*	Creates a stream pattern from a stream file (*.str*) in the script directory.
stopmilling	Halts milling.
waitformill	Pauses script execution until completion of the **millnowait** command.
xsect *x1*, *y1*, *x2*, *y2*	Creates a cross-section pattern. ($x1$, $y1$) and ($x2$, $y2$) define the upper left (or NW) and lower right (or SE) corners, respectively. This pattern type mills a set of five parallel boxes of different sizes and depths, resulting in a ramped depth profile, with the deepest part along the upper edge of the defined area. The system assigns an index value for this pattern and sets the value in the result variable **patkey**.

Imaging Commands

TABLE 5 Imaging Commands

Command	Description
getimageppm *scale*	Returns a scaling factor, the number of pixels per mm (not micron), in the specified variable. Use this value to determine magnification for stage moves. More pixels per mm mean the magnification is higher.
getscan	Returns the current scan parameters. The following result variables are created: **xstart** and **ystart** Upper left corner of the scan **xsize** and **ysize** Lower right corner of the scan **scandwell** Dwell time per pixel **filterstate** Milling filter state (not supported) See **grabframe** for more information about these variables.
grabframe [*x1*, *y1*, *x2*, *y2*, *scanres*, *dwell*, *filter*]	Grabs a frame using either the specified parameters or the current scan parameters as specified by xT or a previous **grabframe** command if no parameters are specified. *x1*, *y1* are the pixel locations of the upper left (or NW) corner of the scan. The upper left (NW) corner of the full frame is defined as 0, 0. *x2*, *y2* are the pixel locations of the lower right (or SE) corner of the scan. *scanres* determines the scan resolution (*x* by *y*, in pixels): 0 256 x 221 (low resolution) 1 512 x 442 (medium resolution) 2 1024 x 884 (high resolution) *dwell* sets the dwell time per pixel, in nanoseconds. This can be set to any value greater than 300 ns. The corresponding values for the selected scan speeds are: fast = 500 ns, medium = 800 ns, and slow = 2400 ns. *filter (*Not supported) determines whether the milling filter is used for imaging.. 0 The system selects the optimal filter based on the scan parameters to provide the best image. 1 The system uses the current filter (image quality may suffer). Use 1 if you need fast processing when combining imaging and milling. See *"Capturing Images" on page 22*.
grabnframes *n*[, *x1*, *y1*, *x2*, *y2*, *scanres*, *dwell*, *filter*]	Integrates *n* frames. *n* must be set to a value selectable from the xT choices in Integrate\Number of Frames. If you set *n* to 1 (one) then this command performs the same as the **grabframe** command.
saveimage *number*	Saves the current image to a file named <*number*>.*bmp*. The file is located in the same directory as the parent script. *number* can have up to sixteen digits. For example: `saveimage 16` saves the image to a file named *16.bmp*.
saveimagewithprefix *number*	Saves an image with format <prefix><number>.bmp (<number> with three digits), example:myprefix000.bmp. **Note:** The prefix cannot be set with a script command.
saveimagewithzeros *number, #zeros*	Saves an image with format < number>.bmp (<number> with #zeros digits), example: 000.bmp

Cognex VisionPro Tool Commands

TABLE 6 Cognex VisionPro Tool Commands

Command	Description
match *bitmapfile* [*xbitmap*, *ybitmap* [, *xulc*, *yulc*, *xdim*, *ydim*]]	Loads a stored bitmap file and finds the best match on the currently displayed image. Note that there is no comma after the file name. (The file is located in the same directory as the parent script, or the path can be specified with the file name.) The parameters *xbitmap* and *ybitmap* range from 0 to 1, with (0, 0) defined as the upper left corner of the bitmap and (1, 1) as the lower right. These are scale parameters, not image coordinates in microns. Entering 0.5, 0.5 returns the location of the center of the bitmap, just as if you had not entered these parameters. You may define a search region within the image. *xulc* and *yulc* define the upper left corner of the search region. *xdim* and *ydim* define the dimensions of the search region in microns. The command returns the match location (**x**, **y**) **result**, and **score**. Note that other commands, such as **getstagepos**, also set the **x** and **y** result variables. **score** is 0 if the match failed. It is between 0 and 1 if the match was successful, where a higher value indicates a better match. The result variable **result** is set to 1 if the match was successful and to 0 if the match failed. See *"Finding a Feature" on page 18*.

Control Beam Commands

Commands of the form **set...** change the state of the system. Commands of the form **get...** return current state information from the system in either result variables or specified variables. Although the commands listed in *Table 7* are control beam commands, they also affect the primary beam. The primary beam is used for live imaging. The control beam is controlled by the MUI and other controls where the same function, e.g., focus, can be applied to either beam. The control beam is normally the same as the primary beam, but it can be changed by the user. For example, when milling with the ion beam, you can grab frames with the electron beam. Changing the control beam from I to E allows the beam shift of the electron beam to be adjusted or the focus to be changed.

TABLE 7 Control Beam Commands

Command	Description
getbeamrot *beamrot*	Returns the scan (beam) rotation of the control beam in degrees in the specified variable.
getbeamshift	Returns the beam shift in microns, in the result variables **xbeam** and **ybeam**.
getctrlbeam *column*	Returns the control beam in the specified variable: 0 indicates the ion column and 1 the electron column.
getmag	Returns the image magnification of the control beam in the result variable **mag**.

TABLE 7 Control Beam Commands

Command	Description
getprimbeam *column*	Returns the primary beam in the specified variable: 0 indicates the ion column and 1 the electron column.
getstig	Returns the column stigmation values in the result variables **xstig** and **ystig**.
setbeamrot *beamrot*	Sets the scan (beam) rotation of the control beam to the specified value.
setbeamshift *xshift*, *yshift*	Sets the beam shift of the control column in microns. The result variable **result** is set to 1 if the beam shift is successful and to 0 if the requested beam shift is out of range for the column.
setctrlbeam *column*	Sets which beam has control: 0 specifies the ion column and 1 the electron column.
setmag *mag*	Sets magnification for the control beam. Specify *mag* as you would in the xT Beam Control page.
setprimbeam *column*	Sets the primary beam: 0 specifies the ion column and 1 the electron column.
setstig *xstig*, *ystig*	Sets the stigmation of the control beam to the specified values.

Ion Column Commands

Unless specified otherwise, voltages are in volts.

TABLE 8 Ion Column Commands

Command	Description
getaperture	Returns the ion column aperture in the result variable **aperture**. The lowest current is defined as 1.
getapertureinfo *current, spotsize, voltage, aperture*	Returns the current (in pA), spotsize (in Angstrom), and HV (in kV) for the specified ion column aperture.
getbeamcurrent	Returns the ion column blanking current, in pA, in the result variable **bcurrent**.
getcolumnpressure	Returns the ion column pressure (in Pascal) in the result variable **pressure**. A returned value of -1 indicates a pressure below the readable scale, and a value of -2 indicates that the pressure is at atmosphere (off).
getfocus	Returns the ion column working distance (in mm) in the result variable **focus**.
gethtstate *state*	Returns the Ion Beam Source and Ion Beam HT state. The *state* values are: 1 Source off; HT off 1 Source on; HT off 2 Source on; HT on
gethv *voltage*	Returns the HV, in kV, of the ion column in the specified variable.
getnumberofionapertures *apertures*	Gets the number of ion column apertures on the given system

TABLE 8 Ion Column Commands

Command	Description
setaperture *aperture*	Sets the ion beam aperture to a specified preset number. *aperture* must be a value from 1 to *max*, where 1 is the first aperture on the menu (typically 1 pA) and *max* the last aperture (depends on aperture strip).
setfocus *distance*	Sets the working distance (in mm) for the ion beam.
sethtstate *state*	Sets the Ion Beam Source and Ion Beam HT state. The *state* values are: 1 Source on; HT off 2 Source on; HT on
sethv *voltage*	Sets the HV of the ion column to the specified value (in kV).

Electron Column Commands

TABLE 9 Electron Column Commands

Command	Description
autofocus	Adjusts the working distance to obtain the sharpest image. The sharpest image is determined by grabbing several frames and adjusting the focus after each frame. The primary beam must be the electron beam (else the command results in an error). The primary beam can be set with the **setprimbeam** command. Returns the result in the result variable **focusresult:** 1 if **autofocus** was successful or 0 if **autofocus** failed.
egetcolmode *mode*	Returns the mode of the electron column in the specified variable: 0 (Mode I), 1 (Mode II, UHR)
egetcolumnpressure *uppercpres*, *lowercpres*	Returns the upper and the lower chamber pressures (in Pascal) in the specified variables. A returned value of -1 indicates a pressure below the readable scale (< 4e-10 mbar), and a value of -2 indicates that the pressure is at atmosphere (off).
egethv *voltage*	Returns the HV value, in kV, in the specified variable. If the beam is off, it returns 0.
egethvonoff *state*	Returns the state of the electron column in the specified variable: 1 indicates HV is on and 0 if it is off.
egetminmaxwd *min*, *max*, *mode*	Returns the minimum and maximum working distances in millimeters for the current mode (Mode I or Mode II). The value of Mode is ignored.
egetspot *espot*	Returns the spotsize (an integer) of the current electron beam.
egetwd *ebeamwd*	Returns the current working distance for the electron beam in the specified variable.
esethv *voltage*	Sets the HV to the specified value (in kV).
esethvonoff *state*	Sets the electron column HV: 1 to set HV on or 0 to set it off.
esetspot *espot*	Sets the electron beam spotsize to a specified preset number. *espot* must be in the range 1 to 7.

Stage Commands

The stage coordinates x, y, and z are in microns; r and t are in degrees.

TABLE 10 Stage Commands

Command	Description
getstagepos	Reads the current stage position and returns the stage coordinates in the result variables **x**, **y**, **z**, **r**, and **t**.
stageccrot *angle*	Performs a relative compucentric rotation, angle is in degrees.

stagemove xyzrt, *xabs*, *yabs*, *zabs*, *rabs*, *tabs*	Moves the stage to an absolute position. You may specify from one to five positions. Script execution waits until stage move is complete.
	For example:
	`stagemove xy, 1000, 500`
	moves the stage in X and Y.
	`stagemove xrt, 1000, -180, 30`
	moves the stage in X, R, and T.

Detector Commands

The commands below always apply to the selected detector.

TABLE 11 Detector Commands

Command	Description
autocb	Finds the optimal level for detector contrast for image matching. The result variable **cbresult** is set to 1 if **autocb** was successful or 0 if **autocb** failed.
getcb	Returns the detector settings in the result variables **c** (contrast) and **b** (brightness). Note that the units are different from those in the UI! In general, use **getdetbrightness** and **getdetcontrast**.
getdetbrightness *brightness*	Returns the detector brightness value in the specified variable. Note that the units are different from those in the UI!
getdetcontrast *contrast*	Returns the detector contrast value in the specified variable. Note that the units are different from those in the UI!
getselectdet *detector*	Determines the current detector and returns the result in the specified variable. The most commonly used *detector* values are: 14 ETD 5 STEM 3 CDEM 1 TLD (through-the-lens)
setcb *c*, *b*	Sets contrast and brightness. In general, use **setdetbrightness** and **setdetcontrast**. Note that the units are different from those in the UI!
setdetbrightness *brightness*	Adjusts the detector brightness to the specified value. Note that the units are different from those in the UI!
setdetcontrast *contrast*	Adjusts the detector contrast to the specified value. Note that the units are different from those in the UI!

Miscellaneous Commands

TABLE 12 Miscellaneous Commands

Command	Description
# [*comment*]	A comment statement begins with a # followed by any information you wish to enter. Insert comments at the beginning of the script and wherever necessary to explain a script to a user looking at it later. For example, at the beginning of a script, you might put: `# script for realigning the laser` Comments are ignored during execution of the script.
dbgstarttimer	Starts the script timer.
dbgstoptimer	Stops the script timer. Calculates the elapsed time in milliseconds between execution of the commands **dbgstarttimer** and **dbgstoptimer**. Returns the value in the result variable **dbgtimer**.
dialog *#buttons*, [“*b1name*”, …] **“***title***”,** *string*	Displays a dialog box. Script execution waits until a button is selected. The result variable **dresult** indicates which button the user pressed. When the number of buttons is set to zero, the dialog box has OK and CANCEL buttons. Clicking OK sets **dresult** to 1, and clicking CANCEL sets it to 2. Alternately, the first parameter specifies the number of buttons displayed in the dialog box, up to a maximum of four. The names define the label of each of the buttons. The result variable **dresult** indicates which button was clicked in the dialog box. If *n* buttons are created, the value returned is 1 to *n*. The *title* text displays in the title bar of the dialog box. *string* can be any combination of text set in double quotes and variables, separated by commas. See *“Producing a Dialog Box” on page 23*.
dump	Writes all defined variables into the log file. This command is useful for debugging scripts.
getchamberpressure *pressure*	Returns the chamber pressure (in Pascal) in the specified variable. A returned value of -1 indicates a pressure below the readable scale, and a value of -2 indicates that the pressure is at atmosphere (off).
goto end	The command **goto end** exits the script.
goto *label*	Branches the script to the next line that begins with *label*. The command Complex control behaviors are possible using **goto** and **if**.
gotosleep	Puts the microscope into sleep mode, i.e. HV off for ion and electron beam, and ion soure off.
hardclear	Deletes all defined variables from the script.
if (*expression*) *statement*	Executes the statement if the expression within the parentheses is true. The statement can be a command or the setting of a variable. Note that the word “then” is not used.
logtime	Writes the current date and time into the log file.

TABLE 12 Miscellaneous Commands

Command	Description
printlog *string*	Prints a string to the log file chosen within the automation control application or specified by **setlogfile**. The string ends with a carriage return and line feed. *string* can be any combination of text set in double quotes and variables, separated by commas. See *"Recording Data" on page 22*.
printlognolf *string*	Prints a string to the log file chosen within the automation control application or specified by **setlogfile**. The string does not end with a carriage return and line feed. *string* can be any combination of text set in double quotes and variables, separated by commas. See *"Recording Data" on page 22*.
run *scriptfile*	Executes a child script, then returns execution to the parent script. The child and parent scripts are in the same directory.
setlogfile *extension*, *append*	Creates a log file named *autolog* with the extension set to the value of *extension*. The file name extension must be a number from 0 to 99. The file is saved in the same directory as the parent script. *append* indicates whether new log information should be appended to the existing file or whether it should overwrite any existing information: 0 indicates append and 1 indicates overwrite. For example: `setlogfile 23, 0` creates a logfile named *autolog.23*. Information written to this logfile (with **printlog**) is appended to information already in the file.
sigfigs *width*, *precision*	Defines the formatting used in printing numbers with the **printlog** command. *width* sets the total number of digits to display, including the decimal point. *precision* defines the number of digits to display after the decimal. For example, `sigfigs 6, 2` would format a number as 111.11.
sleep *milliseconds*	Pauses script execution for the specified time.
timeofday	Inserts the current hour, minute, and second into the log file.
wakeup	Wakes up the microscope, i.e. HV on for ion and electron beam, and ion soure on.

Math Commands

Use the commands in *Table 13* to perform calculations.

NOTE:

Angles are expressed in radians. To convert radians to degrees, use the formula: degree = radian x (180 ˜ 3.142).

TABLE 13 Imaging Commands

Command	Description
acos *variable*, *input*	Stores the arc cosine of *input* in the specified variable.
asin *variable*, *input*	Stores the arc sine of *input* in the specified variable.
sin *variable, input*	stores the sine of input in the specified variable.
atan *variable*, *input*	Stores the arc tangent of *input* in the specified variable.
cos *variable*, *input*	Stores the cosine of *input* in the specified variable.

sqrt *variable*, *input*	Stores the square root of *input* in the specified variable. For example:
	`sqrt answer, 25`
	`dialog 0, “Sq. Root of 25”, “Result = ”, answer`

tan *variable*, *input*	Stores the tangent of *input* in the specified variable.

Variables

AutoScript uses several types of variables to control the xT system functions and the flow of the script. Some variable types are intrinsic to the control application, while others are associated with specific scripting commands.

Variable names and their values can be saved to a log file (see *"Recording Data" on page 22*) or displayed in a dialog box (see *"Producing a Dialog Box" on page 23*).

If one script calls another script with the **run** command, the child script inherits all the variables created within the parent script. Likewise, all the variables created by the child script are passed back to the parent script.

For example, a main processing script can set parameters that define a search area for a recognition script. The recognition script can then return the position of a located feature to the main script, and this position can then be passed to a milling script.

Using Expressions

You can use expressions to assign values to variables, and a parameter can be an expression.

Valid expression symbols include:

+, -, *,/, =, <, >, <> (not equal), >=, and <=.

For example:

if ((x1 + x2) > 4) result = 0

Operations are performed left to right without regard to traditional order of operations. If an expression includes more than one mathematical operation, indicate any variation in the left-to-right order of operation with parentheses.

For example:

value = 5 + (3 * 7)

You can also write expressions that compare values, such as

A = (B < C)

The result is 0 if the comparison is false, or 1 if it is true. In this example, if B is less than C, then A is assigned a value of 1. If B is not less than C, then A is assigned a value of 0.

User-Defined Variables

These variables are explicitly defined by the user with any mathematical expression, including expressions that use other variables. For example, the variable height can be declared with the statements:

```
height = 5.0
or
height = 0.5 * (leftx + rightx)
```

When creating variables, keep the following in mind:

- Variable names can be of any length.

- Variables are not case-sensitive; however, write variables in lower case.
- Avoid special characters.

All user-defined variables except those that begin with *save* are reset to zero every time a script is executed. Any variable that is not explicitly assigned a value is assumed to have a zero value.

Variables that begin with the prefix *save*, such as *savelastx,* are not reset to 0. The values of these variables remain defined until the program running the script is reset or closed. This feature can be useful for many applications, such as setting a flag after a pattern has downloaded or saving information from the previously executed script.

For example, the following scripting lines load a milling pattern on the first execution of the script and bypass the pattern loading on subsequent executions:

```
if (saveisloaded = 1) goto startmilling
patternfile testpat.str
saveisloaded = 1
startmilling:
```

Result Variables

Some AutoScript commands create result variables and set their values. The result variables are described in the descriptions of the AutoScript commands.

NOTE

See also *Using Expressions,* ***15***

Intrinsic Variables

Other variables exist that are not created within a script nor set by an AutoScript command. As noted, some of these can be explicitly declared.

TABLE 14 Intrinsic Variables

Variable	Description
dacppm	Represents the image recognition resolution in pixels per micron, according to the formula 4096/HFW, where HFW is the horizontal field width of the image. The variable is set when a script is first started and changed in the script only by the **setmag** command. If you manually change magnification during a pause in the script, the value of **dacppm** will be incorrect. Update the variable by using **getmag** followed by **setmag mag**.
posnum	Indicates the block position currently being processed. Used by AutoFIB. This value can only be read, not explicitly set.
threshold	Sets the recognition threshold to be used by the **match** command. The default value is 0.5, but it can be explicitly set to any value between 0 and 1. With a threshold of 0.5, the **match** command will generally find matches with a score of around 0.25 and above. Reducing the threshold results in a longer search time that returns matches on a wider range of images. Increasing the threshold results in a shorter search that returns a match on only the better images.

Automation Program Variables

The following table lists the variables set by each of the automation programs.

TABLE 15 Automation Program Variables

Variable	Description
visualmatch	Indicates if the Cognex match window will be displayed when a **match** command is executed. Programs that run AutoScript scripts, like AutoFIB and RunScript, change the value of **visualmatch** automatically based on menu and dialog choices the user makes. You can also set the value of **visual-match** within a script..

Writing a Script

Example Scripts

To write a script, begin by fully understanding the actions you want the system to perform, then determine which AutoScript commands should be used.

No beginning or ending commands are required, though including comments is recommended for clarity. Commands must be one per line, and they are case-sensitive. Be careful of spacing and punctuation.

This section contains examples of the most commonly used commands. For a full list of commands, see *"Commands" on page 4*.

Finding a Feature

The **match** command is the basis of the entire image recognition process for the Cognex VisionPro tool. It takes an image, compares a bitmap file to all or part of that image, then returns the location of the best match.

The units of the search field and match results are microns, and the origin is at the center of the screen. This coordinate system is the same as that used by the milling patterns, so it is a straightforward task to pass data from recognition scripts to milling scripts.

The bitmap files are assumed to be in the standard Windows *.bmp* format. By default, the directory used is the same as that of the main processing script. You can also explicitly declare a relative path. Bitmaps can be generated with Windows Paint™ or similar programs, or they can be cut and pasted from stored FIB images.

Ensure that the bitmap used is appropriate for the magnification of the images and that the system magnification is properly calibrated.

Forms of the Match Command

The **match** command can be used in three forms. The best match is an area.

NOTE	The syntax of the **match** command is described in Table 6, "Cognex VisionPro Tool *Commands," on page 8*.

- Search over the entire image, find the best match, and return the location of the center of the bitmap in the result variables **x** and **y**. For example:

```
match corner.bmp
```

- Search over the entire image, find the best match, and return in the result variables **x** and **y** the location defined by the specified point. For example:

```
match corner.bmp 0,0
```

The specified point (in this example, 0, 0) defines the **x** and **y** values to return to the script. The specified point is defined by scale factors, from 0 to 1. For example, specifying 0.5, 0.5 returns the location of the center of the bitmap, just as if you had not specified these parameters.

- The user defines a search region within the image. Search over the defined region, find the best match, and return in the result variables **x** and **y** the location defined by the specified point.

For example:

```
match corner.bmp 0,0,0,25,25,25
```

The search region is independent of system magnification. If you define a search region that extends beyond the boundaries of the current image, the search region is automatically set to the image boundary.

Examples of the Match Command

Figure 1 shows examples of the **match** command. The screen image in this example is a light background behind a dark rectangular feature. The goal is to locate the upper right corner of the dark rectangle using the bitmap image *corner.bmp*.

Set the **threshold** variable before using the **match** command. Threshold sets the sensitivity of the match command, specifying the allowable amount of difference between the source bitmap and the matched image.

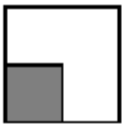
For example:

```
threshold = 0.5
match corner.bmp
```

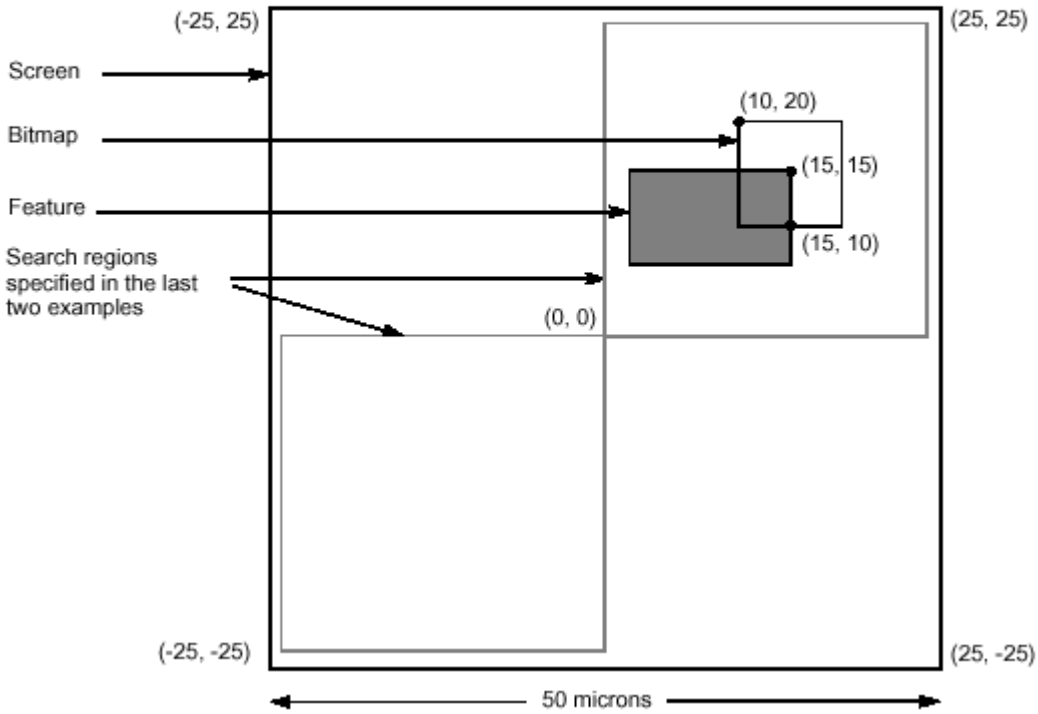
This example searches the entire image and returns the (**x**, **y**) coordinates of the dark rectangle's upper right corner, which matches the center of the bitmap.

The additional examples illustrate how changing parameters produces different results. Note that different forms of the **match** command can return the same point.

FIGURE 1 Examples of the Match Command



To the left is the bitmap image for which the examples below will attempt to find a match: *corner.bmp*.
Desired dimensions: 10 microns/side
For 512 x 512 image scan resolution, 50 micron HFW, pixels/side = 10 x (512 / 50) = ~102



Command	Result
match corner.bmp	Returns location (15, 15).
match corner.bmp 0, 0	Returns location (10, 20).
match corner.bmp 0.5, 0.5	Returns location (15, 15).
match corner.bmp 0.5, 1.0	Returns location (15, 10).
match corner.bmp 0.5, 0.5, 0, 25, 25, 25	Searches upper right quadrant. Returns location (15, 15).
match corner.bmp 0.5, 0.5, -25, 0, 25, 25	Searches lower left quadrant. Match command fails.

Automating Milling

AutoScript can mill a pattern two ways: by explicitly defining the dimensions of a milling pattern, or by loading a stream or pattern file directly from disk.

Defined Dimensions

The milling pattern types available with AutoScript include many of those supported by the software: line, box, open box, cross section, circle, bitmap and stream file. (See the xT system documentation for additional information about milling.)

To perform a milling operation using an AutoScript script:

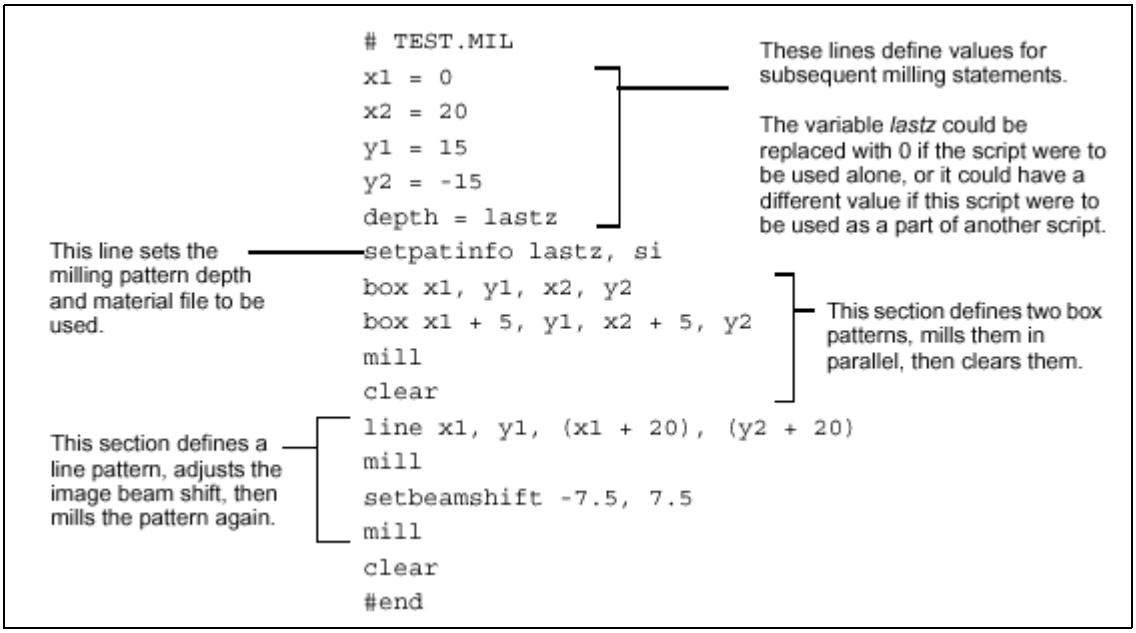
1 Define the milling depth.

2 Define the pattern area.

3 Mill the pattern.

If additional patterns are to be milled, the existing defined patterns may need to be cleared. See *"Patterning and Milling Commands" on page 8* for information on patterning and milling commands. Below is a simple example of a milling script.

FIGURE 2 Milling Script Example



NOTE:

Precede milling commands with the **clear** command to ensure accurate milling.

If patterns are not cleared, every call of the **mill** command mills them.

The milling patterns supported by AutoScript are the same as those supported by xT, except cleaning cross section.

However, scripts can be written to perform the same function but with the additional flexibility of controlling the milling of each individual line.

The actual milling time for patterns within AutoScript is determined automatically by the depth and application file selected by the **setpatinfo** command and by the ion beam current being used.

Stream or Pattern Files

Stream files are ASCII or binary files that contain the screen locations of each point to be milled. In contrast to patterning commands, such as box, they offer more flexibility with regard to pattern types and beam scanning, but they must be generated with an additional program, such as *PSConvert.exe*. (See the *PSConvert User's Guide*, PN 27432.). Stream files (*.str*) can be loaded with the **streamfile** command.

The stream file mills for a specific time, independently of beam current. The file loads over a specific location on the screen independently of magnification, but once loaded, will change size if the magnification is changed.

Pattern files (*.ptf*) saved from xT (File > Export > Patterns) can be loaded with the **patternfile** command.

Capturing Images

The **grabframe** command grabs an image at the current scan parameters. For example, to grab a full frame corresponding to the medium resolution, slow scan setting of the software and using the imaging filter (not supported), the command would be:

```
grabframe 0,0,512,442,1,2400,0
```

This command would scan the full screen, as shown on the left in *Figure 3* below.

If you have set a reduced area scan using the **grabframe** command, then single scans and continuous imaging will also scan that selected area. Restore normal scanning by selecting Scan > Reduced Area (F7) menu item or the toolbar button in xT.

Figure 3 shows examples of how different dimensions affect the size of the scan area.

These examples assume medium resolution, fast scan, no filter. If the pattern does not produce a good image, widen the pattern or increase the dwell time.

NOTE	The syntax of the **patternfile** and **streamfile** commands is described in *Table 4,* "Patterning and Milling *Commands," on page 4*.

NOTE	The syntax of the **grabframe** command is described in *Table 5,* "Imaging Commands," on *page 7*.

Recording Data

The **printlog** command writes a string of text and variables to the log file name chosen within the automation control application. Use the **setlogfile** command to specify a different log file name.

For example, the script commands:

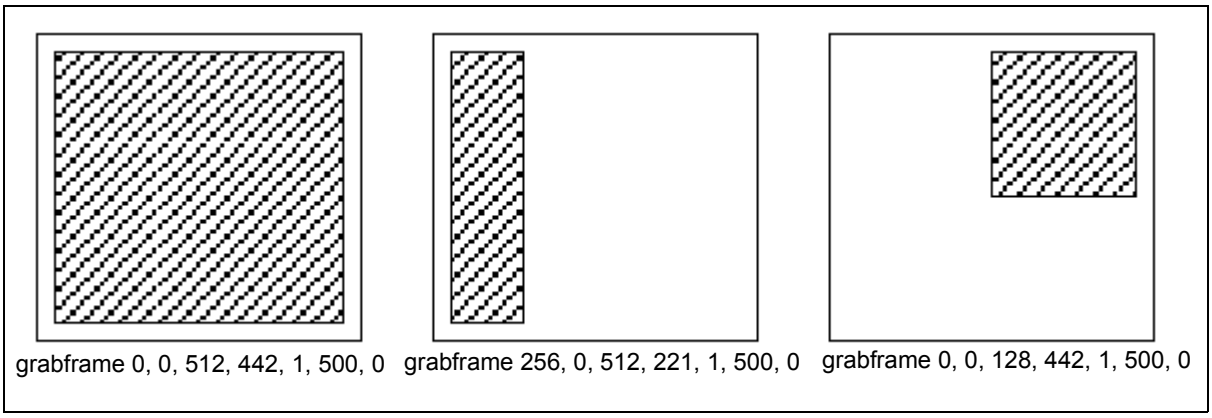
```
sigfigs 6,2
printlog "head = ",headnum,"x = ",polex,"y = ",poley
```

will append the log file with a line of the form:

```
head = 35.00 x = 3.44 y = -0.44
```

The automation control applications, for example AutoFIB, add extra log file entries each time the script is run.The log file will also contain error messages.

FIGURE 3 Examples of Scan Dimensions



Producing a Dialog Box

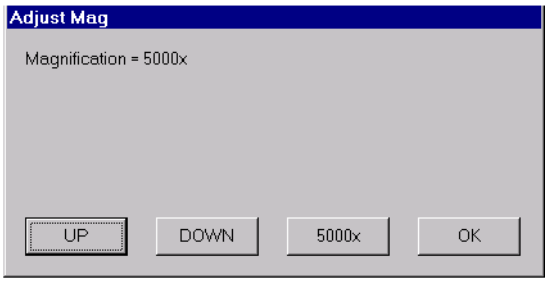
The **dialog** command is similar to **printlog**, except that the string is displayed within a dialog box that appears in the interface. Script execution waits until one of the dialog buttons is clicked. The result variable **dresult** indicates which button was clicked.

The output string is of the same format as used in **printlog**, with the additional feature that a carriage return within a set of quotes will start a new line of text within the displayed dialog box.

Following is an example of using a **dialog** command within a script. This script allows the user to adjust the magnification up or down, or to reset it to 5000X.

```
# MAGSET.PSC
setmag 5000
loop:
getmag
grabframe
dialog 4,“UP”,“DOWN”,“5000x”,“OK”,“Adjust Mag”, “Magnification = ”,mag, “x”
if (dresult = 1) setmag = mag + 1000
goto mag_is_set
if (dresult = 2) setmag = mag - 1000
goto mag_is_set
if (dresult = 3) setmag = 5000
goto mag_is_set
if (dresult = 4) goto continue
goto mag_is_set
mag_is_set:
goto loop
continue:
```

This script produces the following dialog box:



- Clicking UP sets **dresult** to 1. The magnification is increased by 1000 X, and a new dialog displays with the updated value.

- Clicking DOWN sets **dresult** to 2. The magnification is decreased by 1000 X, and a new dialog displays with the updated value.
- Clicking 5000x sets **dresult** to 3. The magnification is reset to 5000 X, and a new dialog displays with the updated value.
- Clicking OK sets **dresult** to 4. The magnification is unchanged, and the script jumps to the "continue" label.

Troubleshooting

NOTE	The syntax of the **dialog** command is described in Table 12, "Miscellaneous *Commands," on page 12*.

Script failure is usually the result of errors in imaging, programming, or pattern recognition. Always look in the log file when confronted with a failure. You will frequently be able to diagnose the problem using information from the log file. Below are some common causes of script failures.

Causes of Imaging Errors

- Ion beam poorly focused
- Incorrect image contrast
- Incorrect scan resolution or scan speed
- Incorrect ion beam aperture

Causes of Program File Errors

- Recognition script is in different directory than processing script
- Recognition or milling script files nonexistent
- Recognition script calls incorrect bitmap files
- Milling script calls incorrect stream file

Causes of Recognition Errors

- Match threshold too low or too high
- Bitmap created for a different sample type
- Bitmap created for a different scan resolution
- Match area incorrectly placed
- Bitmap too large for match area
- Value of **result** incorrectly set to 0 (zero) by script

The outcome of a script depends on the final value of the variable **result**: 1 specifies success, 0 specifies failure. This value is determined by the last matching command executed, unless a new value of **result** has been explicitly assigned. If a script does not contain matching commands nor the line "result=1", it fails.

Causes of "Invalid ASCII" Error Messages

- Character combinations in the script, such as ALT 181
- Duplicate labels
- Unbalanced quotes

Tips

Clearing Patterns

In scripts, use the **clear** command before defining patterns. This ensures that only the patterns just defined will be milled.

Improving Milling Performance

- To increase milling rate, increase beam current. Beware, however, that a higher beam current produces gentler side wall slopes and less accurate side wall edges.
- Reduce the milling box size or box depth if processing requirements do not require those parameters to be a set size.
- Improve the milling profile by milling at a shorter working distance, unless use of the GIS along with tilt would interfere.
- Tilt the sample while milling to improve the slope profile of the milled structure.

Enhancing Recognition Capabilities

- To reduce grabframe time, reduce the grabframe area.
- To increase scan rate, reduce grabframe scan resolution (pixel counts) and pixel dwell time (pixel averaging). Resolution may drop and image noise may increase.
- To reduce matching time, reduce the search area.
- To reduce the number of bitmaps required, include more complexity in each bitmap.
- To improve recognition success rate, reduce brightness to minimum and use only contrast control to maximize image contrast. Be careful not to oversaturate the image and lose edges.

An Alphabetical list of xT Commands

acos	**getstig**
asin	**goto**
atan	**gotosleep**
autocb	**grabframe**
autofocus	**grabnframes**
bitmap	**hardclear**
box	**if**
circle	**insert**
clear	**insertbyname**
cos	**line**
dbgstarttimer	**logtime**
dbgstoptimer	**match**
deletepattern	**mill**
dialog	**Millnowait**
dump	**obox**
egetcolmode	**patternfile**
egetcolumnpressure	**polygon**
egethv	**printlog**
egethvonoff	**printlognolf**
egetminmaxwd	**retract**
egetspot	**retractbyname**
egetwd	**run**
esethv	**saveimage**
esethvonoff	**saveimagewithprefix**
esetspot	**saveimagewithzeros**
esetwd	**setaperture**
getaperture	**setapplicationfile**
getapertureinfo	**setbeamrot**
getbeamcurrent	**setbeamshift**
getbeamrot	**setcb**
getbeamshift	**setctrlbeam**
getcb	**setdetbrightness**
getchamberpressure	**setdetcontrast**
getcolumnpressure	**setfocus**
getctrlbeam	**sethtstate**
getdetbrightness	**sethv**
getdetcontrast	**setlogfile**
getfocus	**setmag**
getgisfromname	**setparallelmode**
gethtstate	**setpatbeam**
gethv	**setpatinfo**
getimageppm	**setpatternrect**
getismilling	**setprimbeam**
getmag	**setrotation**
getnumberofionapertures	**setstig**
getparallelmode	**sigfigs**
getpatbeam	**sin**
getpatternrect	**sleep**
getprimbeam	**sqrt**
getrotation	**stageccrot**
getscan	**stagemove**
getselectdet	**stopmilling**
getstagepos	**streamfile**

tan	
timeofday	
waitformill	
wakeup	
xsect	

Index

Symbols
12
(x, y) result *8*

A
a cross-section pattern *6*
a dialog box *12*
acos *14*
Alphabetical list of Commands *26*
aperture *9*
aperture strip *10*
application file *22*
arc cosine *14*
arc sine *14*
arc tangent *14*
asin *14*
atan *14*
autocb *11*
AutoFIB *2*
autofocus *10*
Automating Milling *21*
Automation *17*
AutoScript *1*

B
b *11*
bcurrent *9*
beam shift *8*, *9*
bitmap *4*
blanking current *9*
box *4*

C
c *11*
Capturing Images *22*
causes of script failures *24*
cbresult *11*
chamber pressure *12*
chamber pressures *10*
child *1*
child script *13*
circle *4*
clear *4*, *21*
Clearing Patterns *25*
Cognex VisionPro tool *18*
Cognex VisionPro Tool Commands *8*
column pressure *9*
column stigmation *9*
commands *1*
comment statement *12*
Complete List of Commands *4*
compucentric rotation *11*
control beam *8*
Control Beam Commands *8*
cos *14*
cosine *14*
creating variables *15*
current *9*

D
dacppm *17*
dbgstarttimer *12*
dbgstoptimer *12*
dbgtimer *12*
debugging scripts *12*
deletepattern key *4*
detector brightness *11*
Detector Commands *11*
detector contrast *11*
dialog *12*, *23*
dresult *12*, *23*
dump *12*
dwell time *7*

E
egetcolmode *10*
egetcolumnpressure *10*
egethv *10*
egethvonoff *10*
egetminmaxwd *10*
egetspot *10*
egetwd *10*
Electron Column Commands *10*
error messages *22*
esethv *10*
esethvonoff *10*
esetspot *10*
Example Scripts *18*
expression *1*

F
FEI Automation Control Applications *2*
file name extension *13*
filterstate *7*
focus *9*
focusresult *10*
formatting used in printing numbers *13*

G
getaperture *9*

getapertureinfo *9*
getbeamcurrent *9*
getbeamrot *8*
getbeamshift *8*
getcb *11*
getchamberpressure *12*
getcolumnpressure *9*
getctrlbeam *8*
getdetbrightness *11*
getdetcontrast *11*
getfocus *9*
getgisfromnam *4*
gethtstate *9*
gethv *9*
getimageppm *7*
getismilling *4*
getmag *8*
getnumberofionapertures *9*
getparallelmode *4*
getpatbeam *4*
getpatternrect *4*
getprimbeam *9*
getrotation *4*
getscan *7*
getselectdet *11*
getstagepos *11*
getstig *9*
goto *12*
goto end *12*
gotosleep *12*
grabframe *7*, *22*
grabnframes *7*

H
hardclear *12*
HV *9*, *10*

I
if *12*
image magnification *8*
image recognition *18*
image recognition resolution *17*
Imaging Commands *7*
insert *5*
insertbyname *5*
Intrinsic Variables *17*
ion beam aperture *10*
Ion Beam HT stat *9*
Ion Beam HT state *10*
Ion Beam Source *9*, *10*
ion column apertures *9*
Ion Column Commands *9*

L
line *5*
log file *12*
logtime *12*

M
mag *8*
magnification *7*, *9*
match *8*, *18*, *19*
Math Commands *14*
mathematical operation *15*
mill *5*, *21*
milling filter *7*
Milling Performance *25*
milling profile *25*
milling rate *25*
millnowait *5*
Miscellaneous Commands *12*
mode of the electron column *10*

O
obox *5*

P
parameters *1*
parent script *1*, *13*
patkey *4*, *5*, *6*
pattern rotation *6*
patternfile *5*
polygon *5*
posnum *17*
prefix save *16*
primary beam *8*
printlog *13*, *22*, *23*
printlognolf *13*
Producing a Dialog Box *23*
PSConvert.exe *22*

R
r *11*
Recognition Capabilities *25*
recognition success rate *25*
recognition threshold *17*
Recording Data *22*
reduced area *22*
result *9*, *24*
Result Variables *16*
retract *5*
retractbyname *5*
run *13*
Running Scripts *2*
RunScript *2*

S
saveimage *7*
saveimagewithprefix *7*
saveimagewithzeros *7*
scan (beam) rotation *8*, *9*
scan resolution *7*
scandwell *7*
score *8*
script *1*
Script failure *24*
Script Naming Conventions *2*
Script Structure *1*
script time *12*
script timer *12*
search region *19*
setaperture *10*
setapplicationfile *5*
setbeamrot *9*
setbeamshift *9*
setcb *11*
setctrlbeam *9*
setdetbrightness *11*
setdetcontrast *11*
setfocus *10*
sethtstate *10*
sethv *10*
setlogfile *13*, *22*
setmag *9*
setparallelmode *5*
setpatbeam *5*
setpatinfo *5*, *22*
setpatternrect *5*
setprimbeam *9*
setrotation *6*
setstig *9*
sigfigs *13*
sleep *13*
sleep mode *12*
slope profile *25*
Software Syntax *3*
spotsize *9*, *10*
sqrt *14*
Stage Commands *11*
stage position *11*
stageccrot *11*
stagemove *11*
state of the electron column *10*
statement *1*
stigmation *9*
stopmilling *6*
Stream or Pattern Files *22*
streamfile *6*, *22*

T
t *11*
tan *14*
threshold *17*, *19*
timeofday *13*

U
User-Defined Variables *15*
Using Expressions *15*

V
value *1*
variable *1*
Variables *1*
visualmatch *17*

W
waitformill *6*
wakeup *13*
working distance *9*, *10*
working distances *10*

X
x *11*
xbeam *8*
xsect *6*
xsize *7*
xstart *7*
xstig *9*

Y
y *11*
ybeam *8*
ysize *7*
ystart *7*
ystig *9*

Z
z *11*